

Learning-Augmented Page Replacement Under Workload Shift

CSE-307: Operating Systems — Term Paper, Track 1

Student Name: _____ ID: _____

Abstract

Page replacement is an important memory-management problem in operating systems because the number of pages that can stay in physical memory is limited. This project compares FIFO, LRU, and Optimal (Belady) page replacement and adds a small learned component that predicts a useful page to evict. The experiment uses a synthetic access trace whose behavior changes halfway through. The first half is locality-heavy and sequential, while the second half contains random and bursty accesses. The main goal is to observe how page faults change when the workload changes and whether a learned policy can adapt using simple historical features.

1. Problem Framing

Classical page-replacement policies make different assumptions about which page will be useful again. FIFO uses the arrival order, LRU uses recent usage, and Optimal uses future knowledge. A real operating system cannot know the future, so Optimal is treated as a theoretical benchmark. The project asks whether a lightweight learned rule can use information available from the recent access history to make a better eviction choice when the workload changes.

2. Objectives

The implementation follows the Track 1 requirements: (1) implement FIFO, LRU, and Optimal; (2) train a lightweight classifier to predict a good eviction candidate using recency and frequency; (3) generate a workload shift from locality/sequential access to random/bursty access; and (4) compare page-fault behavior before and after the shift.

3. Implementation

The experiment uses four memory frames and a 600-reference synthetic trace. The first 300 references mainly use a small sequential/local working set. The final 300 references alternate short bursts with a wider random page range. This creates an intentional change in access behavior.

Classical algorithms

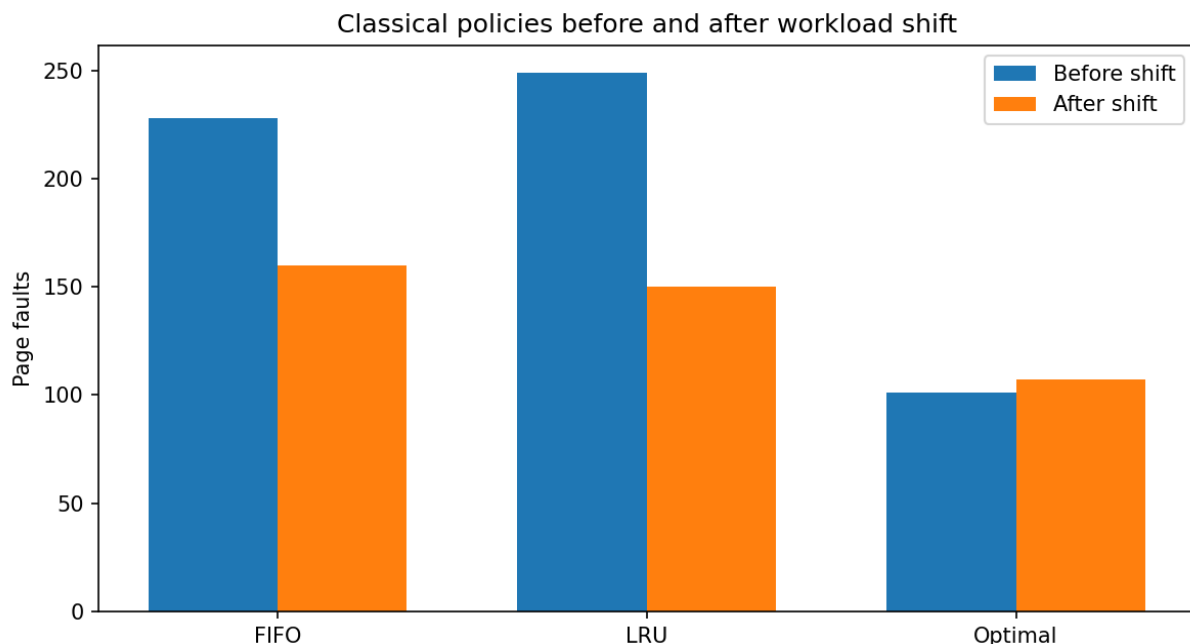
FIFO: evicts the page that has been in memory for the longest time. **LRU**: evicts the page that has not been referenced for the longest time. **Optimal**: evicts the page whose next reference is farthest in the future. Optimal is used as a benchmark because its future information is not normally available to an OS.

Learned component

The learned component is a small binary logistic-regression classifier implemented directly in Python. For every page currently in memory, it calculates four features: normalized recency, frequency in the last 30 references, frequency in the last 100 references, and an indicator that the page has become very old. During training, the Optimal policy supplies the target eviction candidate. During testing, the classifier sees only historical information and selects the candidate with the highest predicted probability of being the optimal eviction.

4. Experimental Results

Policy	Total faults	Before shift	After shift
FIFO	388	228	160
LRU	398	249	150
Optimal	205	101	107
Learned	285	138	147



In this run, FIFO produced 388 page faults, LRU produced 398, Optimal produced 205, and the learned policy produced 285. The learned eviction classifier had an eviction-candidate accuracy of 0.423.

5. Analysis of the Workload Shift

The workload change matters because the useful behavior of a page can change over time. In the first half, repeated local references make recent-history information useful. After the shift, the access range becomes wider and includes random/bursty behavior, increasing the number of replacements needed by the classical policies. The before/after table should therefore be read as a comparison of the same algorithms under two different access regimes rather than as a universal ranking.

The learned policy recorded 0.012 average confidence margin on correct eviction decisions and 0.009 on incorrect decisions in this run. These values give a simple check of whether the model's confidence is related to correctness. Because the classifier is trained from synthetic traces and the feature set is intentionally small, the result should be interpreted as an educational experiment rather than a production memory-management method.

6. Limitations

The workload is synthetic, the memory size is fixed at four frames, and the page population is small. The learned model also uses labels derived from Optimal, so it is learning to imitate a theoretical policy rather than discovering an independent replacement objective. More traces, different frame counts, and real memory-reference traces would be useful for a stronger evaluation.

7. Conclusion

This project demonstrates how a classical operating-system heuristic can be combined with a lightweight learned component. FIFO, LRU, and Optimal provide clear reference points, while the learned policy uses recent access information to choose an eviction candidate. The deliberate workload shift shows why a policy that works under one access pattern may behave differently after the pattern changes. The experiment also shows the importance of evaluating an adaptive method on both sides of a workload shift instead of reporting only one overall number.

8. AI Assistance Disclosure

AI assistance was used for implementation guidance, debugging support, documentation, and report drafting. The student should run the supplied program, inspect the generated results, and revise the final analysis where necessary so that the submitted experimental claims reflect the student's own understanding and work.